

Numerical Design Optimization Project 2 Report

6687

October 17 2025

Executive Summary

The objective of this project was to optimize the shape of a wing spar of a long-endurance telecommunications unmanned aerial vehicle, or UAV, such that it can withstand expected load conditions while minimizing the weight of the spar. The spar is a 15 meter long circular annulus made of a carbon fiber composite, with a density of $1600 \frac{kg}{m^3}$, Young's modulus of 70 GPa, and ultimate tensile and compressive strength of 600 MPa, although only half of the spar was analyzed, giving it an effective length of 7.5 meters. The spar could not have an inner radius of less than 1 centimeter, an outer radius of more than 5 centimeters, or a thickness of less than 2.5 millimeters. The maximum expected load on the spar occurs during a 2.5g maneuver, where the mass of the aircraft is 500 kilograms. The optimization was required to improve upon the weight of a nominal design by at least 60%, where the nominal design has a constant outer radius of 5 centimeters, and inner radius of 4.35 centimeters, giving it a total mass of 13.26 kilograms.

In order to model the spar a number assumptions were taken such as planar symmetry, smooth cross-sections, normality, bending only internal strain energy, linear deformation effects, and constant property, isotropic materials. Volume was taken as the optimization objective, with inner and outer radii being the design variable. The constraints were the dimensions of the radii, as well as the stress on the spar.

A finite element discretization was used to solve for the stress along the spar and the spar volume. Matlab's Fmincon method was then used to perform the optimization with the active-set

algorithm. Functions were developed to calculate the stress at each node along the spar and the total volume. These functions also calculated the Jacobian of stress and gradient of volume with a complex step differentiation and provided it to Fmincon. A convergence study showed that the optimal number of elements was 70. The optimization successfully identified a design which satisfies all constraints and creates optimized mass of 5.0243 kilograms, representing a 62% decrease in mass over the nominal design.

The optimization was successful, although the proposed design should not be taken as the final spar. The assumptions this design is built off of underestimate stress. Specifically, the assumption that the fiber composite is isotropic should be replaced with more accurate ultimate tensile and compressive stress figures. Additionally, analysis into stress concentrations that arise when attaching the spar to the plane should be examined.

Methodology

Optimizing the shape of a wing spar such that it can withstand expected stresses while minimizing spar weight implies an optimization problem with volume minimization as the objective, inner and outer radii of the spar as the design variables, and maximum stress as a constraint. Additional constraints were placed upon the maximum inner and outer radii of the spar, as well as the spar thickness to ensure that the result of the optimization is manufacturable. Following identification of the optimization objective, design variables and constraints, the following assumptions were taken to formulate a Euler-Bernoulli based mathematical model of the wing spar: the spar exhibits planar symmetry, smooth cross sections, and normality. Additionally, it is assumed that internal strain energy only accounts for bending moment deformations, as well as that deformations are small enough to neglect nonlinearities, and that the carbon fiber composite is elastic and isotropic. These assumptions are generally standard in

Euler-Bernoulli analysis, although they do pose some physical concerns. First, planar symmetry assumes no major manufacturing errors in the wing spar, as well no significant asymmetric loading during flight. Next, the smooth cross section assumption breaks down at joints or attachments on the spar, which will be necessary to couple the spar to the air frame and wing. Finally, carbon fiber composites are not isotropic, and the reported tensile and compressive ultimate strength is only valid in directions aligned with the carbon fibers.

Given the assumptions listed above, the stress along the beam can be found with Eq (1), and then compared to the maximum strength of the carbon fiber to evaluate the feasibility of a design.

$$\sigma_{xx}(x) = -z_{max}E \frac{d^2w}{dx^2} \quad (1)$$

Where z_{max} is the maximum height of the spar, E is Young's modulus, w is the displacement of the spar, and x is the distance along the spar. Given the total load on the spar is constant, Eq (2) can be used to solve for the displacement.

$$\frac{d^2}{dx^2} (EI_{yy} \frac{d^2w}{dx^2}) = q \quad (2)$$

Where I_{yy} is given with Eq (3), and q is given by Eq (4).

$$I_{yy} = \frac{\pi(D^4 - d^4)}{64} \quad (3)$$

$$q = \frac{2.5g}{2} \quad (4)$$

Where D is the outer diameter of the spar, d is the inner diameter of the spar, and g is the acceleration due to gravity. The volume of a design can be calculated with Eq (5).

$$volume = \pi L(R^2 - r^2) \quad (5)$$

Where L is the length of the spar, R is the outer radius of the spar and r is the inner radius of the spar. Volume can then be multiplied by density to obtain the mass of the spar.

Given the models above, a finite element discretization was taken to solve for the stress along the spar as well as the total volume. A convergence study based on the nominal design of the spar was conducted to determine the optimal number of elements.

Matlab's `fmincon` method was leveraged to perform the optimization routine. Specifically, the active-set algorithm was used with `GradObj` and `GradConst` options on. The active-set algorithm iteratively identifies and enforces the subset of constraints that are active at a potential optimum, solving a series of quadratic subproblems to move toward a minimizer. The `GradObj` and `GradConst` options imply that the objective function and constraint function both provide `fmincon` the gradient of the functions, rather than relying on `fmincon`'s default finite difference gradient approximation. A complex step based approach was used to calculate these gradients due to its accurate approximation of derivatives without subtractive errors.

Constraints were implemented by creating linear inequality matrix equations and functions as well as bound constraints and including them within the call to `fmincon`. The spar thickness constraint was implemented by constructing the linear inequality matrix equation $r_{in} - r_{out} \leq -0.0025$. The radii bounds were implemented by creating bound constraints where $0.00125 < r_{out} < 0.05$ and $0.01 < r_{in} < 0.0475$. The stress constraint was implemented by creating a constraint function that returns a linear inequality matrix of the normalized stress at each node given by Eq (6).

$$\frac{\sigma}{\sigma_{max}} - 1 \leq 0 \quad (6)$$

Results

The convergence study was constructed to evaluate the quality of the finite element mesh by evaluating the stress at the tip of the spar. Analytically, the stress at the tip of the spar should equal 0, so comparing the stress at the final node of the spar to zero provides a good insight to the quality of the mesh. The study was conducted on a uniform spar of 5 cm outer diameter and 4.35 cm inner diameter. The result of the convergence study is given in Figure 1 below.

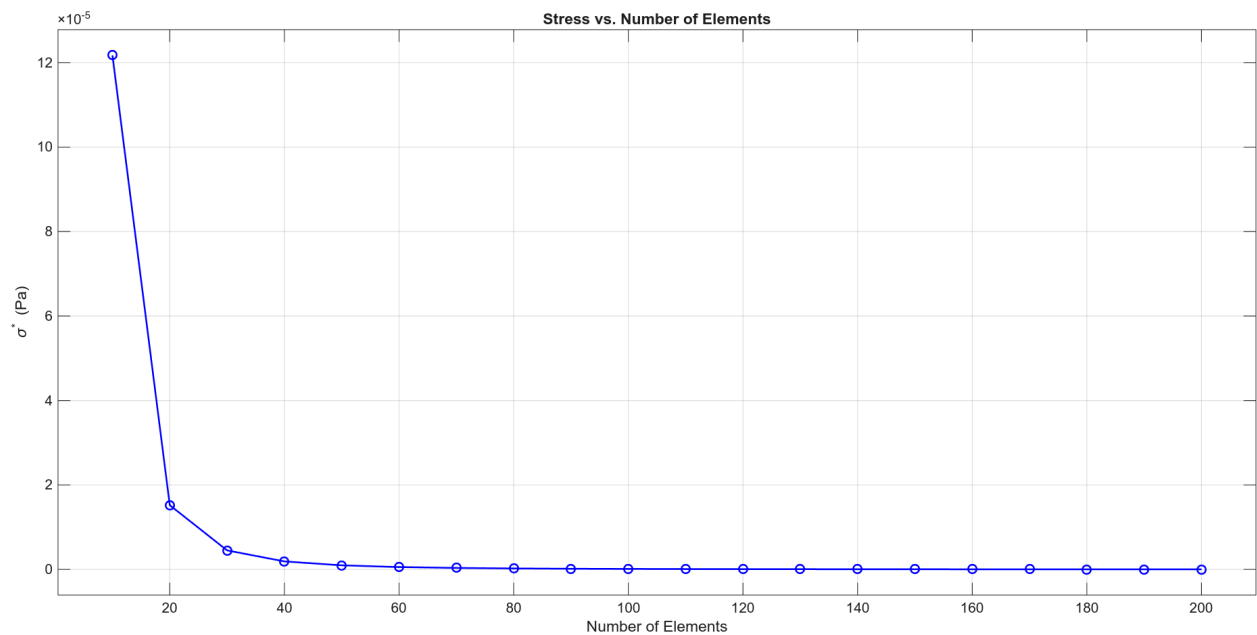


Figure 1: Convergence Study

Clearly the accuracy of the mesh improves quickly, with minimal improvements in accuracy beginning to occur at about 70 elements. Accordingly, 70 elements were taken to produce the optimized design. The initial design was set to be a uniform spar of 5 centimeter outer diameter and 3 centimeter inner diameter. This optimization produced an initially thick spar, which became thinner along the spar, as shown in Figure 2. This makes sense since stress is the highest at the root, and therefore extra reinforcement must be concentrated there. After this, stress will decrease until it equals approximately zero at the tip. Accordingly, the spar decreases its

thickness along the length of the spar until it reaches the minimal allowable thickness to decrease the volume of the spar.

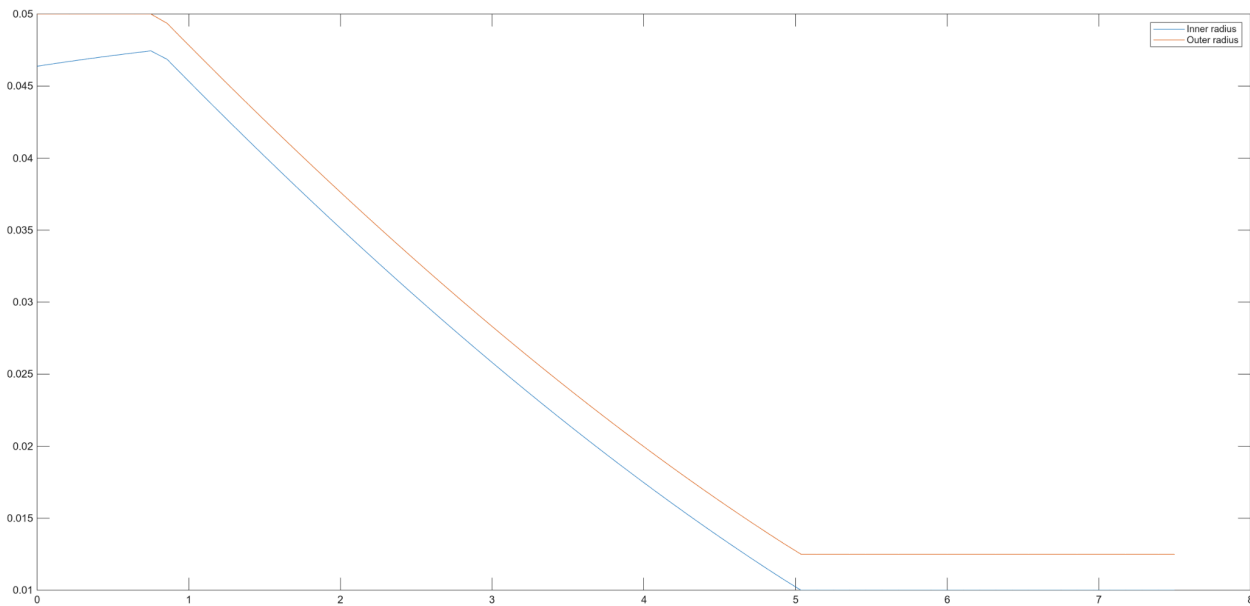


Figure 2: Optimized Spar Design

The mass of the optimized spar is 5.0243 kg, representing a 62% weight reduction over the nominal design. Evidence of convergence is given in Figure 3.

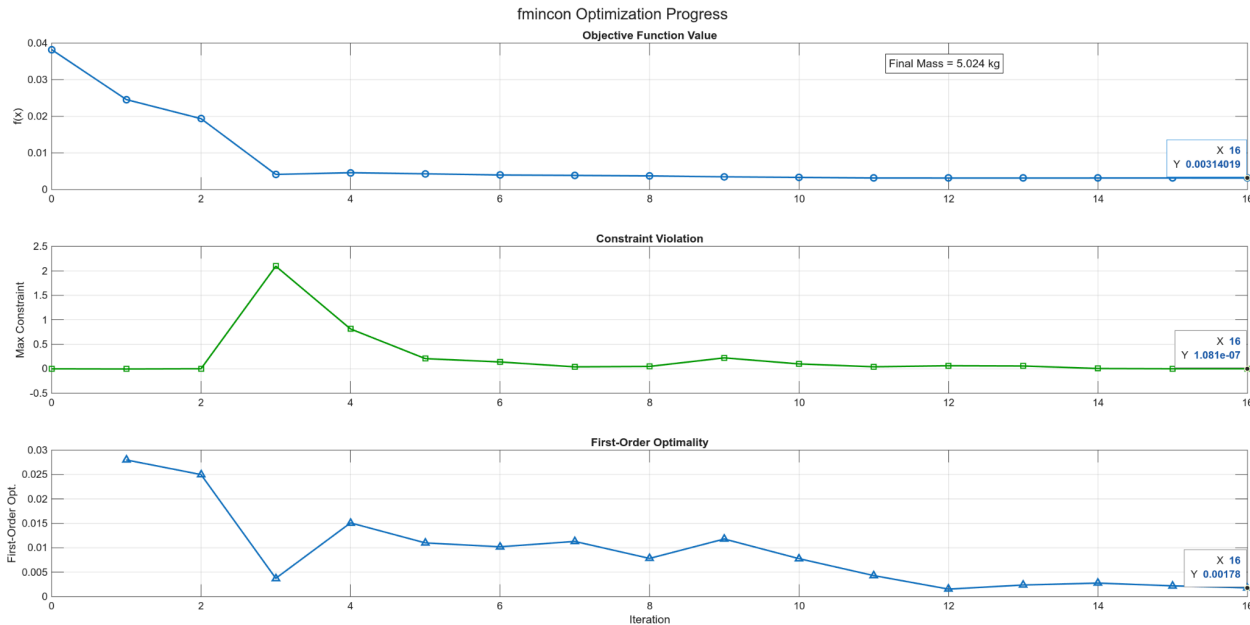


Figure 3: Convergence to Minimizer

Figure 3 illustrates the minimization of the objective function in the top sub-plot. The maximum constraint moves to zero in the second sub-plot, indicating no constraints are being violated. First order optimality is seen reaching zero in the last sub plot, indicating the identification of a local minimizer.

Conclusions

Overall the optimization was a success. A successful and feasible design was created such that all constraints are met and a local minimizer is identified. However, it should be noted that this design should not be taken as the final product. Some of the assumptions underestimate the allowable stress, especially that the carbon fiber composite is isotropic. Additionally, considerations into how the spar will integrate with the rest of the airframe should be made, especially at the root, as this will induce stress concentrations at the location of maximum stress.

Appendix

A run_opt.m

```
clear all
close all
% define variables
Nelem = 70;
nvar = (Nelem+1)*2;
E = 70e9;
L = 7.5;
q_total = 2.5*500*0.5*9.8; % given total load (scalar)
xnodes = linspace(0, L, Nelem+1)';
force = calc_load(xnodes, L, q_total);
%define initial design
r0 = zeros(nvar,1);
r0(1:2:end) = 0.05; % initial outer radius
r0(2:2:end) = 0.03; % initial inner radius
% define constraints
[Aineq, bineq, lb, ub] = ineq(r0);
%fmincon
options = optimset('GradObj', 'on', 'GradConst', 'on', 'Display', 'iter',
'Algorithm', 'active-set');
[x, fval] = fmincon(@(r)obj(r, L), r0, Aineq, bineq, [], [], lb, ub,
@(r)constrain(r, L, force, E, Nelem), options);
[ro, ri] = get_radrii(x);
plot(xnodes, ri);
mass = fval*1600
hold on
plot(xnodes, ro);
legend('Inner radius', 'Outer radius')
```

B constrain.m

```
function [cineq, ceq, Jineq, Jeq] = constrain(r, L, force, E, Nelem)
% Inputs:
% r: array of design variables
% L: length of spar
% x: node locations along spar
% q_total: total force on spar
% E: Young's Modulus
% Nelem: Number of elements
% Outputs:
% cineq: inequality constraint
% ceq: equality constraint --> not used
% Jineq: Jacobian inequality constraint transposed
% Jeq: Jacobian equality constraint transposed
% ceq and Jeq not used
ceq = []; Jeq = [];
```

```

% Find cineq
cineq = subcon(r);
% Find Jineq
Jineq = zeros(size(r, 1), size(cineq, 1)); % dimensions are #dv x #constraints
h = 1e-30;
for i = 1:size(r, 1)
    rc = r;
    rc(i) = rc(i) + complex(0, h);
    Jineq(i, :) = imag(subcon(rc))/h;
end
function [c] = subcon(dv)
[zmax, ~] = get_radii(dv);
Iyy = get_moment(dv);
u = CalcBeamDisplacement(L, E, Iyy, force, Nelem);
[sigma] = CalcBeamStress(L, E, zmax, u, Nelem);
sigmamax = 600e6;
c = (sigma./sigmamax) - 1;
end
end

```

C obj.m

```

function [f, g] = obj(r, L)
% This function calculates the total volume of the spar
% Inputs:
% r - vector containing all radii. Format is ri followed by ro
% L - length of spar
% Outputs:
% f - a scalar representing the total volume of the spar in m^3
% g - the gradient of the objective function
f = subobj(r);
g = zeros(size(r, 1), 1);
h = 1e-30;
for i = 1:size(r, 1)
    rc = r;
    rc(i) = rc(i) + complex(0, h);
    g(i) = imag(subobj(rc))/h;
end
function [fval] = subobj(dv)
% This calculates the volume of the spar
% Inputs: dv - the design variables, inner and outer radii of the
% spar
% Output:
% fval - the volume
[ro, ri] = get_radii(dv);
volume = 0;
dx = L / (length(ri)); % element length
for j = 1:length(ri)
    area = pi * (ro(j)^2 - ri(j)^2);

```

```

        volume = volume + (area * dx);
    end

    fval = volume;
end
end

```

D ineq.m

```

function [Aineq, bineq, lb, ub] = ineq(r)
% Creates all constraint matrices
% Inputs:
% r - array containing the radii at each node
% Outputs:
% Aineq - Inequality matrix imposing spar thickness and stress values
% bineq - inequality array denoting what the minimum thickness and stress
% lb - lower bound for inner radius
% ub - upper bound for outer radius
nvar = length(r);
n = nvar/2;
nelem = n-1;
Aineq = zeros(nelem+1, nvar);
bineq = zeros(nelem+1, 1);
for i = 1:nelem+1
    index2 = i*2;
    index1 = index2 - 1;
    Aineq(i, index1) = -1;
    Aineq(i, index2) = 1;
    bineq(i) = -0.0025;
end
lb = zeros(nvar, 1);
ub = zeros(nvar, 1);
for j=1:2:nvar
    lb(j) = 0.00125;
    lb(j+1) = 0.01;
    ub(j) = 0.05;
    ub(j+1) = 0.0475;
end

```

E get_radII.m

```

function [ro, ri] = get_radII(r)
% This function extracts the inner and outer radii from a vector of all
% radii. It assumes that radii are listed in pairs with the inner radii listed
% before the outer
% inputs:
% r - a vector containing all the radii
% outputs
% ri - a vector containing all the inner radii

```

```

% ro - a vector containing all the outer radii
ro = r(1:2:end); % Extract inner radii (odd indices)
ri = r(2:2:end); % Extract outer radii (even indices)

```

F get_moment.m

```

function Iyy = get_moment(r)
% Find moment of inertia of spar at each node
% Inputs:
% r - The radius at each node. Format is [ri1 ro1 ri2 ro2 ... rin ron]'
% Outputs:
% Iyy - The moment of inertia at each node
[ro, ri] = get_radii(r);
Iyy = zeros(size(ri, 1), 1);
tol = 1e-10;
for i = 1:length(ri)
    D = ro(i)*2;
    d = ri(i)*2;
    Iyy(i) = (D^4 - d^4)*(pi/64);
    if real(Iyy(i)) < tol
        Iyy(i) = tol;
    end
end
end

```

G calclload.m

```

function [qx] = calc_load(x, L, q_total)
% Calculates load at each x node on spar
% Inputs:
% x: array of node locations
% L: scalar length of spar
% q_total: total load on spar
% Outputs:
% qx: load at each x element
% maximum load at root
qmax = q_total*2/L;
%slope
m = -qmax/L;
qx = zeros(size(x, 1), 1);
for i = 1:size(x, 1)
    qx(i) = subcalc(x(i));
end
function y = subcalc(x)
y = (m*x)+qmax;
end
end

```

H output.txt

below is the output to the optimization of the spar

```
>> run_opt
```

Max Line search Directional First-order

Iter F-count f(x) constraint steplength derivative optimality Procedure

```
0 1 0.0382377 0
1 5 0.0245778 -0.00387 0.25 -0.289 0.028 unreliable
2 10 0.01936 0.001447 0.125 -0.24 0.025 unreliable
3 12 0.00412348 2.096 1 -0.226 0.00374
4 14 0.00459034 0.8137 1 0.0433 0.0151
5 16 0.00428975 0.208 1 -0.00405 0.011 Hessian modified
6 18 0.00397026 0.1398 1 -0.00415 0.0102 Hessian modified
7 20 0.0038504 0.03981 1 -0.00535 0.0113 Hessian modified
8 23 0.00371195 0.04891 0.5 -0.00518 0.00782 Hessian modified
9 25 0.00346828 0.2234 1 -0.00642 0.0118 Hessian modified
10 27 0.00330512 0.09963 1 -0.0062 0.00777 Hessian modified
11 29 0.00315602 0.04116 1 -0.00517 0.0043 Hessian modified
12 31 0.00313757 0.06293 1 -0.00419 0.00153 Hessian modified; unreliable
13 33 0.00312515 0.05734 1 -0.00242 0.00237 Hessian modified; unreliable
14 35 0.00314414 0.005852 1 0.00286 0.00276 Hessian modified; unreliable
15 37 0.00314252 0.0004875 1 -0.000706 0.0022 Hessian modified twice
    16      39 0.00314019 1.081e-07 1 -0.00119 0.00178
```

Hessian modified twice

Active inequalities (to within options.ConstraintTolerance = 1e-06):

lower	upper	ineqlin	ineqnonlin
96	1	9	1
98	3	10	2
100	5	11	3
102	7	12	4
104	9	13	5
106	11	14	6
108	13	15	7
110	15	16	8
112		17	9
114		18	10
116		19	11
118		20	12
120		21	13
122		22	14
124		23	15
126		24	16
128		25	17
130		26	18
132		27	19
134		28	20
136		29	21
138		30	22

140	31	23
142	32	24
	33	25
	34	26
	35	27
	36	28
	37	29
	38	30
	39	31
	40	32
	41	33
	42	34
	43	35
	44	36
	45	37
	46	38
	47	39
	48	40
	49	41
	50	42
	51	43
	52	44
	53	45
	54	46
	55	47
	56	
	57	
	58	
	59	
	60	
	61	
	62	
	63	
	64	
	65	
	66	
	67	
	68	
	69	
	70	
	71	

Local minimum possible. Constraints satisfied.

fmincon stopped because the size of the current search direction is less than twice the value of the step size tolerance and constraints are satisfied to within the value of the constraint tolerance.

mass =

5.0243

I plots.m

```
clear; close all; clc;
% --- Data from fmincon output ---
data = [
    0   1   0.0382377   0           NaN       NaN       NaN;
    1   5   0.0245778  -0.00387   0.25     -0.289    0.028;
    2  10   0.01936     0.001447   0.125    -0.240    0.025;
    3  12   0.00412348  2.096     1.0     -0.226    0.00374;
    4  14   0.00459034  0.8137     1.0      0.0433   0.0151;
    5  16   0.00428975  0.208     1.0     -0.00405  0.011;
    6  18   0.00397026  0.1398     1.0     -0.00415  0.0102;
    7  20   0.0038504   0.03981     1.0     -0.00535  0.0113;
    8  23   0.00371195  0.04891     0.5     -0.00518  0.00782;
    9  25   0.00346828  0.2234     1.0     -0.00642  0.0118;
   10  27   0.00330512  0.09963     1.0     -0.0062   0.00777;
   11  29   0.00315602  0.04116     1.0     -0.00517  0.0043;
   12  31   0.00313757  0.06293     1.0     -0.00419  0.00153;
   13  33   0.00312515  0.05734     1.0     -0.00242  0.00237;
   14  35   0.00314414  0.005852     1.0      0.00286  0.00276;
   15  37   0.00314252  0.0004875     1.0     -0.000706 0.0022;
   16  39   0.00314019  1.081e-07    1.0     -0.00119  0.00178;
];
% --- Extract columns ---
Iter = data(:,1);
fx    = data(:,3);
MaxConstraint = data(:,4);
FirstOrderOpt = data(:,7);
% --- Create figure with three subplots ---
figure('Name','fmincon Optimization Progress','NumberTitle','off');
% --- Plot 1: f(x) ---
subplot(3,1,1);
plot(Iter, fx, '-o', 'LineWidth', 1.5, 'DisplayName', 'f(x)');
ylabel('f(x)');
title('Objective Function Value');
grid on;
% --- Plot 2: Max Constraint ---
subplot(3,1,2);
plot(Iter, MaxConstraint, '-s', 'LineWidth', 1.5, 'Color', [0 0.6 0],
'DisplayName', 'Max constraint');
ylabel('Max Constraint');
title('Constraint Violation');
grid on;
% --- Plot 3: First-Order Optimality ---
subplot(3,1,3);
```

```
plot(Iter, FirstOrderOpt, '-^', 'LineWidth', 1.5, 'DisplayName', 'First-order
optimality');
ylabel('First-Order Opt. ');
xlabel('Iteration');
title('First-Order Optimality');
grid on;
% --- Optional: Display final mass ---
mass = 5.0243; % from run_opt output
subplot(3,1,1);
text(max(Iter)*0.7, max(fx)*0.9, sprintf('Final Mass = %.3f kg', mass), ...
    'FontSize', 11, 'BackgroundColor', 'w', 'EdgeColor', 'k');
% --- Improve layout ---
sgtitle('fmincon Optimization Progress');
```